

Apostila Completa

Tkinter & CustomTkinter

Guia prático com todos os widgets, exemplos e telas ilustradas

Python 3 • GUI para Desktop • Edição expandida

Sumário

Parte I — Tkinter

1. Introdução e instalação	3
2. A janela principal (Tk)	4
3. Gerenciadores de geometria: pack, grid, place	5
4. Label	8
5. Button	9
6. Entry	10
7. Text	11
8. Frame e LabelFrame	12
9. Checkbutton	13
10. Radiobutton	14
11. Listbox	15
12. ttk.Combobox	16
13. Scale (slider)	17
14. Spinbox	18
15. Scrollbar	19
16. Canvas (desenho)	20
17. Menu e Menubar	21
18. messagebox	22
19. filedialog e colorchooser	23
20. Toplevel (múltiplas janelas)	24
21. ttk.Treeview (tabela)	25
22. ttk.Notebook (abas)	26
23. ttk.Progressbar	27
24. Variáveis Tk (StringVar etc.)	28
25. Eventos e bindings	29
26. Estilos com ttk.Style	30

Parte II — CustomTkinter

27. Instalação e CTk()	31
28. Aparência e temas	32
29. CTkLabel e CTkButton	33
30. CTkEntry e CTkTextbox	34
31. CTkFrame e CTkScrollableFrame	35

32. CTkCheckBox e CTkRadioButton	36
33. CTkSwitch	37
34. CTkSlider e CTkProgressBar	38
35. CTkOptionMenu e CTkComboBox	39
36. CTkSegmentedButton e CTkTabview	40
37. CTkImage (imagens responsivas)	41
38. CTkToplevel e diálogos	42
39. Layout com grid em CTk	43
40. Projeto: Tela de Login moderna	44
41. Projeto: Dashboard completo	45
42. Boas práticas e dicas finais	47

Parte I — Tkinter

1. Introdução e instalação

Tkinter é a biblioteca padrão do Python para criação de interfaces gráficas (GUI). Ela já vem embutida na maioria das distribuições de Python, baseada no toolkit Tk (originalmente da linguagem Tcl).

Com Tkinter você consegue criar janelas, botões, campos de texto, menus, gráficos básicos e muito mais — tudo de forma multiplataforma (Windows, Linux e macOS).

Verificar instalação: abra o terminal e execute `python -m tkinter`. Se uma pequena janela aparecer, está tudo certo. Em distros Linux pode ser necessário instalar o pacote `python3-tk`.

Exemplo:

```
# Versão mínima para confirmar instalação
import tkinter as tk
print("Versão do Tk:", tk.TkVersion)

janela = tk.Tk()
janela.title("Teste")
janela.geometry("300x100")
tk.Label(janela, text="Tkinter funcionando!").pack(pady=20)
janela.mainloop()
```

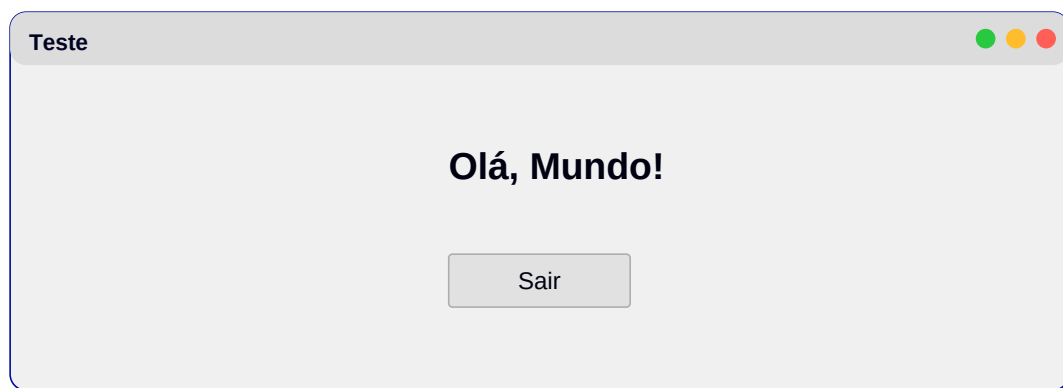


Figura 1.1 — Janela mínima de teste do Tkinter.

2. A janela principal (Tk)

Toda aplicação Tkinter começa com a criação de uma instância de `Tk()`, que representa a janela principal. Após adicionar widgets, chama-se `mainloop()` — o laço que mantém a janela aberta e processa eventos.

Principais métodos da janela: **`title()`** define o título; **`geometry('LxA+X+Y')`** define tamanho e posição; **`resizable(w,h)`** permite ou bloqueia o redimensionamento; **`iconbitmap()`** define o ícone; **`configure(bg=...)`** muda a cor de fundo.

Use **`protocol('WM_DELETE_WINDOW', funcao)`** para interceptar o clique no X de fechar.

Exemplo:

```
import tkinter as tk

def ao_fechar():
    if tk.messagebox.askyesno("Sair", "Deseja realmente sair?"):
        janela.destroy()

janela = tk.Tk()
janela.title("Minha primeira janela")
janela.geometry("400x250+200+100")  # 400x250 px na posição (200,100)
janela.resizable(False, False)
janela.configure(bg="#F4F6FB")
janela.protocol("WM_DELETE_WINDOW", ao_fechar)

tk.Label(janela, text="Olá, Mundo!",
        font=("Helvetica", 18, "bold"), bg="#F4F6FB").pack(pady=40)
tk.Button(janela, text="Sair", width=12, command=janela.destroy).pack()

janela.mainloop()
```

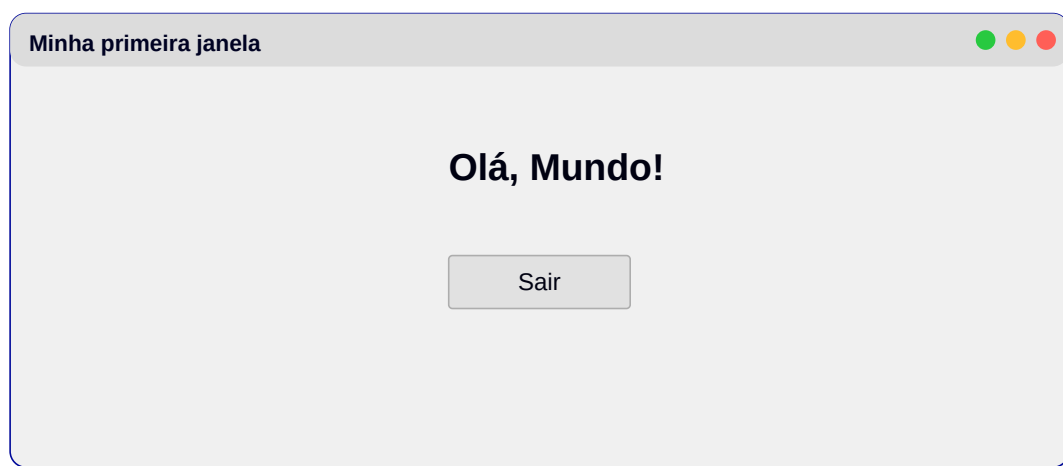


Figura 2.1 — Janela com título, label e botão.

3. Gerenciadores de geometria

Tkinter oferece três gerenciadores: **pack()**, **grid()** e **place()**. Eles definem como cada widget se posiciona dentro do seu pai. Nunca misture **pack** e **grid** no mesmo container — isso trava a interface.

pack: empilha widgets. Aceita *side* (top, bottom, left, right), *fill* (x, y, both), *expand* (True/False) e *padx/pady*.

grid: organiza em linhas e colunas. Aceita *row*, *column*, *rowspan*, *columnspan*, *sticky* ('n','s','e','w' e combinações) e *padx/pady*.

place: posicionamento absoluto ou relativo (*x*, *y*, *relx*, *rely*, *relwidth*, *relheight*). Útil para sobreposições, mas pouco responsivo.

Exemplo:

```
# Exemplo PACK
import tkinter as tk
j = tk.Tk(); j.geometry("420x220")
tk.Label(j, text="Topo", bg="#CDE8FF").pack(side="top", fill="x")
tk.Label(j, text="Base", bg="#FFE0B2").pack(side="bottom", fill="x")
tk.Label(j, text="Left", bg="#C8E6C9").pack(side="left", fill="y")
tk.Label(j, text="Right", bg="#F8BBD0").pack(side="right", fill="y")
j.mainloop()
```

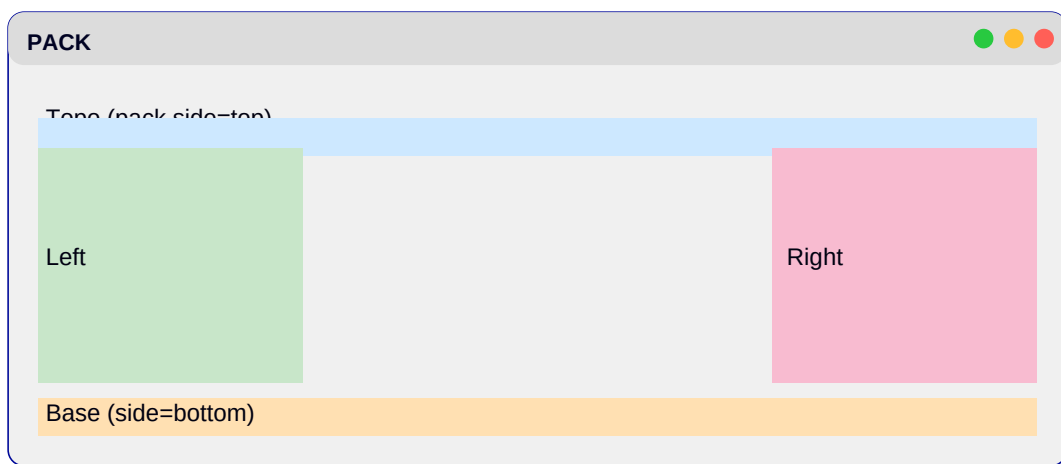


Figura 3.1 — Disposição clássica usando pack().

```
# Exemplo GRID
import tkinter as tk
j = tk.Tk(); j.geometry("360x240")
for r in range(3):
    for c in range(3):
        tk.Label(j, text=f"({r},{c})", relief="ridge",
                  width=10, height=3).grid(row=r, column=c,
                  padx=4, pady=4, sticky="nsew")
for i in range(3):
    j.grid_columnconfigure(i, weight=1)
    j.grid_rowconfigure(i, weight=1)
j.mainloop()
```

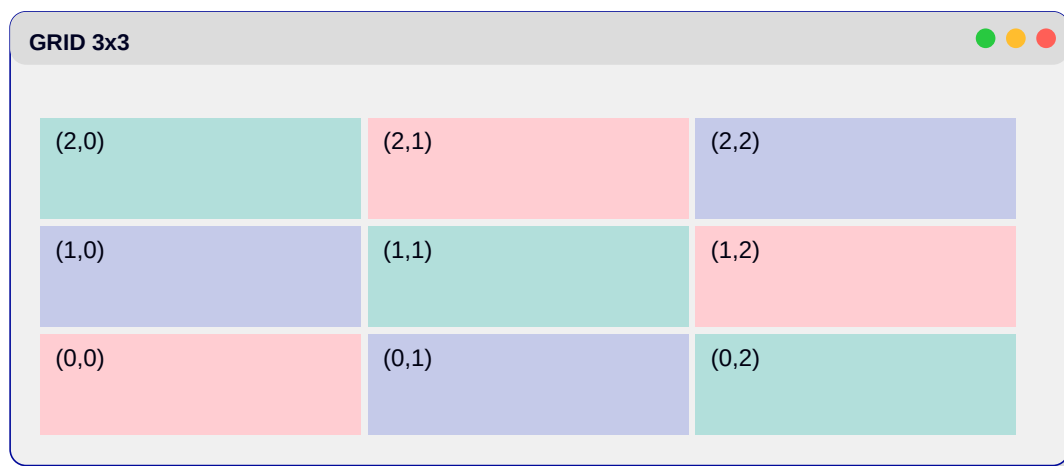


Figura 3.2 — Grade 3×3 com grid() e pesos para expansão.

Comparativo rápido

- Use **pack** para layouts lineares simples (uma toolbar no topo, status na base).
- Use **grid** para formulários e telas tabulares — é o mais utilizado em apps reais.
- Use **place** apenas para sobreposições, fundos absolutos ou efeitos visuais.

Dica: para deixar o conteúdo responsivo no grid use `columnconfigure(i, weight=1)` e `rowconfigure(i, weight=1)`.

4. Label

O **Label** exibe textos curtos, imagens ou ícones. Não recebe input do usuário. Os parâmetros mais usados são:

text, **font**=('Família', tamanho, 'estilo'), **fg** (cor do texto), **bg** (cor de fundo), **image**, **anchor** ('n','s','e','w','center'), **justify**, **wraplength** (quebra de linha automática em pixels).

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.geometry("360x180"); j.title("Labels")

tk.Label(j, text="Título grande",
         font=("Helvetica", 16, "bold"), fg="#0F3460").pack(pady=8)
tk.Label(j, text="Subtítulo em itálico",
         font=("Helvetica", 11, "italic"), fg="#555").pack()
tk.Label(j, text="Texto longo com quebra automática " * 4,
         wraplength=320, justify="left", fg="#222").pack(pady=10)
j.mainloop()
```

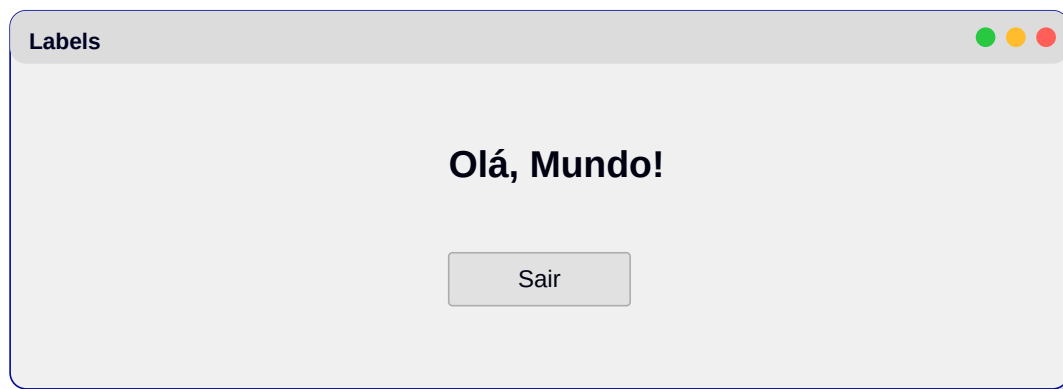


Figura 4.1 — Exemplo de uso do Label.

5. Button

O **Button** dispara uma função quando clicado. Parâmetros principais: **text**, **command** (função a executar), **width/height**, **bg/fg**, **state** ('normal','disabled'), **relief** ('flat','raised','sunken','ridge','groove'), **cursor**.

Use lambda quando precisar passar argumentos: `command=lambda: funcao(arg)`.

Exemplo:

```
import tkinter as tk
def somar():
    label.config(text=f"Total: {int(label.cget('text').split(':')[1]) + 1}")

j = tk.Tk(); j.geometry("260x140"); j.title("Botão contador")
label = tk.Label(j, text="Total: 0", font=("Helvetica", 14, "bold"))
label.pack(pady=12)
tk.Button(j, text="+1", width=10, command=somar,
          bg="#0078D4", fg="white", relief="flat").pack()
tk.Button(j, text="Sair", width=10, command=j.destroy).pack(pady=6)
j.mainloop()
```

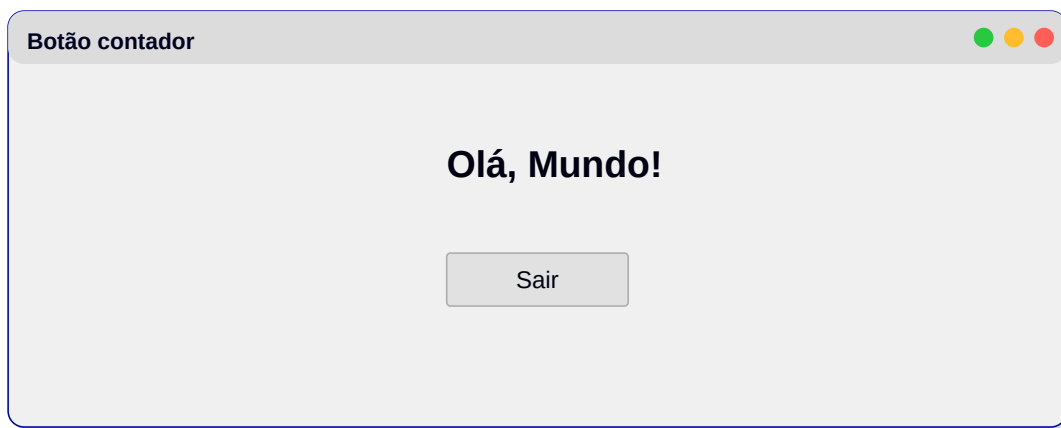


Figura 5.1 — Botão dispara incremento do contador.

6. Entry

O **Entry** é um campo de texto de uma única linha. Métodos importantes: **get()** retorna o texto digitado, **insert(idx,'texto')**, **delete(0,'end')**.

Parâmetros úteis: **show='*' (para senhas)**, **textvariable** (vincula a uma StringVar), **state**, **validate** e **validatecommand**.

Exemplo:

```
import tkinter as tk
from tkinter import messagebox

def entrar():
    if usr.get() == "admin" and pwd.get() == "1234":
        messagebox.showinfo("OK", f"Bem-vindo, {usr.get()}!")
    else:
        messagebox.showerror("Erro", "Credenciais inválidas.")

j = tk.Tk(); j.title("Login"); j.geometry("320x180")
tk.Label(j, text="Usuário:").grid(row=0, column=0, padx=8, pady=8, sticky="e")
tk.Label(j, text="Senha:").grid(row=1, column=0, padx=8, pady=8, sticky="e")
usr = tk.Entry(j, width=22); usr.grid(row=0, column=1, padx=8)
pwd = tk.Entry(j, width=22, show="*"); pwd.grid(row=1, column=1, padx=8)
tk.Button(j, text="Entrar", width=10, command=entrar
        ).grid(row=2, column=0, columnspan=2, pady=12)
j.mainloop()
```

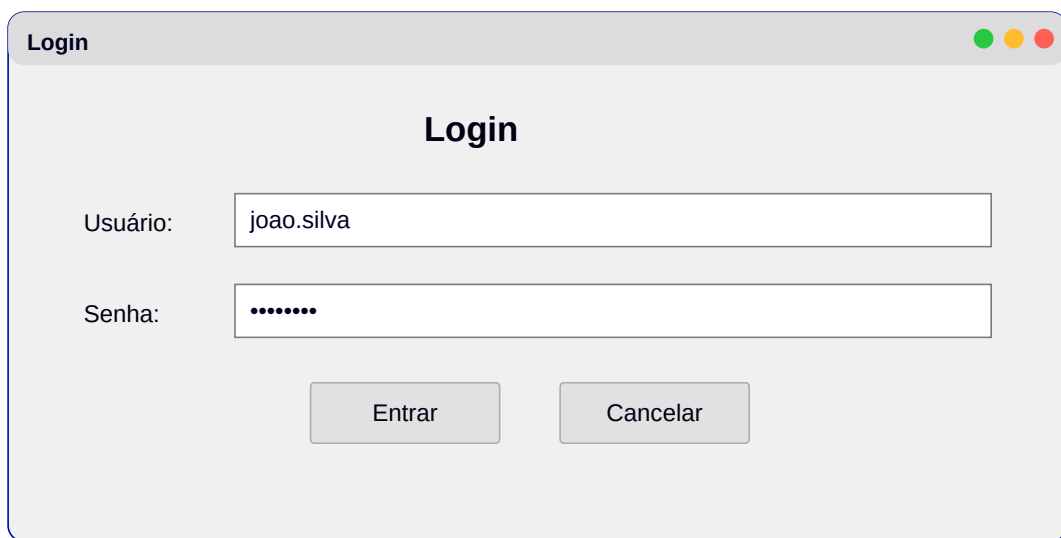


Figura 6.1 — Formulário de login simples.

7. Text

O widget **Text** aceita múltiplas linhas e formatação rica (tags com cores, fontes diferentes, links). É usado em editores, blocos de notas e chats.

Métodos: `insert('end','texto')`, `get('1.0','end')`, `delete('1.0','end')`, `tag_add`, `tag_config`.

Acoplar um Scrollbar é simples: crie a barra, conecte `yscrollcommand=barra.set` e `barra.config(command=texto.yview)`.

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Editor"); j.geometry("480x260")

barra = tk.Scrollbar(j); barra.pack(side="right", fill="y")
txt = tk.Text(j, wrap="word", yscrollcommand=barra.set,
              font=("Consolas", 11))
txt.pack(fill="both", expand=True)
barra.config(command=txt.yview)

txt.insert("end", "# Editor de texto Tkinter\n\n")
txt.insert("end", "def ola_mundo():\n    print('Olá!')\n")
txt.tag_add("kw", "3.0", "3.3")
txt.tag_config("kw", foreground="#0033B3", font=("Consolas",11,"bold"))
j.mainloop()
```

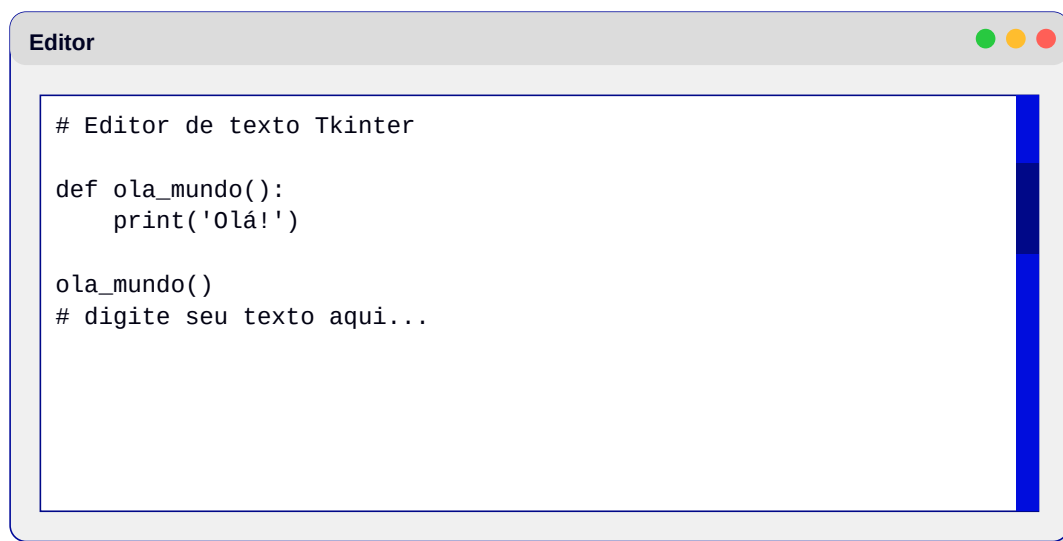


Figura 7.1 — Widget Text com scrollbar acoplada.

8. Frame e LabelFrame

Frame é um container invisível usado para agrupar e organizar widgets — fundamental para layouts complexos.

LabelFrame é igual ao Frame, mas tem uma moldura com título. Parâmetros: **text**, **padx/pady**, **bd** (borda), **relief**.

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Frames"); j.geometry("400x250")

topo = tk.Frame(j, bg="#CDE8FF", height=40); topo.pack(fill="x")
tk.Label(topo, text="Cabeçalho", bg="#CDE8FF",
         font=("Helvetica", 12, "bold")).pack(pady=8)

dados = tk.LabelFrame(j, text=" Dados pessoais ", padx=10, pady=10)
dados.pack(fill="both", expand=True, padx=12, pady=10)
tk.Label(dados, text="Nome:").grid(row=0, column=0, sticky="e")
tk.Entry(dados, width=24).grid(row=0, column=1, padx=6, pady=4)
tk.Label(dados, text="Email:").grid(row=1, column=0, sticky="e")
tk.Entry(dados, width=24).grid(row=1, column=1, padx=6, pady=4)
j.mainloop()
```

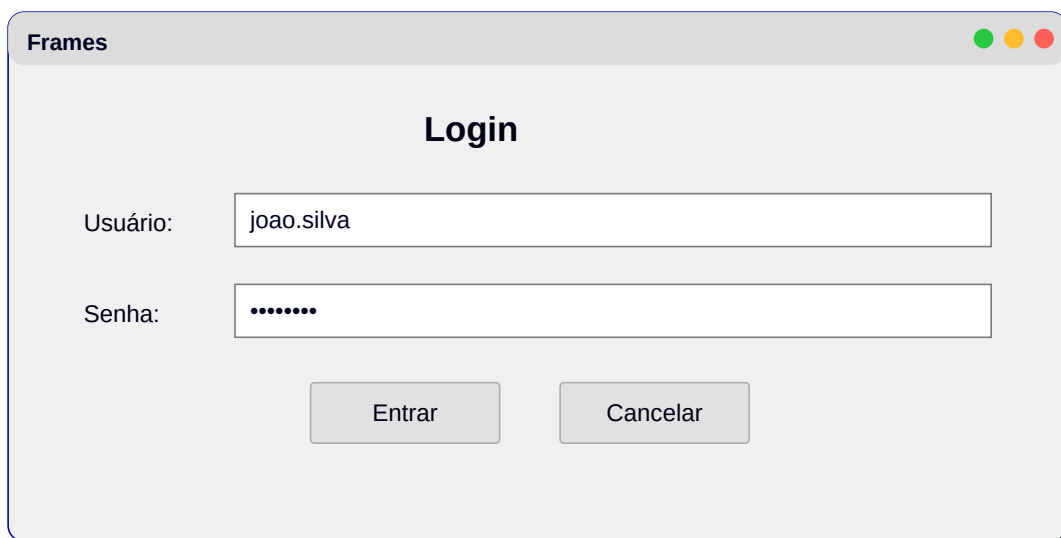


Figura 8.1 — LabelFrame agrupando campos de um formulário.

9. Checkbutton

Permite marcar/desmarcar opções. Geralmente associado a uma **BooleanVar** ou **IntVar**.

Métodos: **select()**, **deselect()**, **toggle()**. Use **variable** para vincular e **onvalue/offvalue** para definir os valores.

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Hobbies"); j.geometry("260x220")

opcoes = ["Programar", "Ler", "Jogar", "Esportes", "Música"]
vars_ = {o: tk.BooleanVar(value=(o in ("Programar", "Ler", "Música"))
    for o in opcoes}

tk.Label(j, text="Selecione seus hobbies:",
    font=("Helvetica", 11, "bold")).pack(anchor="w", padx=10, pady=6)
for o in opcoes:
    tk.Checkbutton(j, text=o, variable=vars_[o]).pack(anchor="w", padx=14)

def mostrar():
    print([o for o, v in vars_.items() if v.get()])
tk.Button(j, text="OK", command=mostrar).pack(pady=8)
j.mainloop()
```

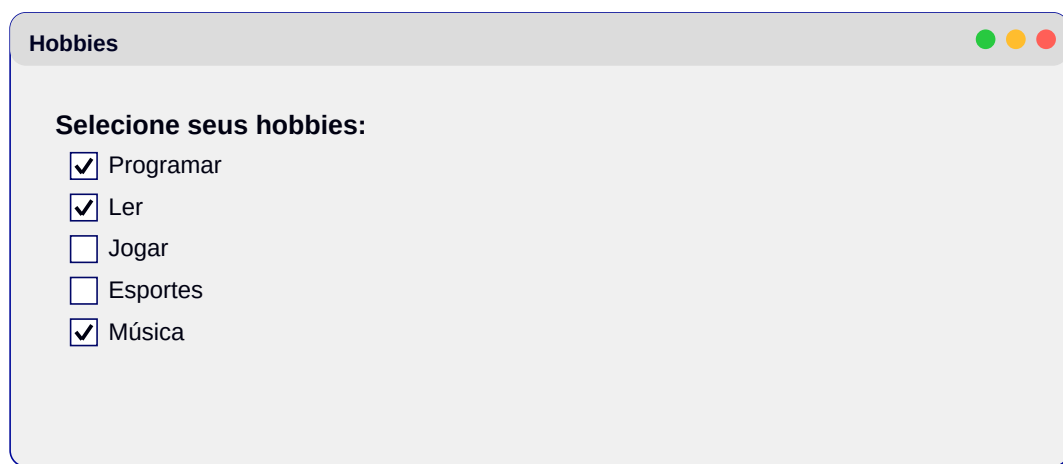


Figura 9.1 — Lista de Checkbuttons com itens selecionados.

10. Radiobutton

Permite escolher **uma única opção** dentre várias. Todos os Radiobuttons relacionados devem compartilhar a mesma variável (IntVar/StringVar) e ter um **value** distinto.

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Nível"); j.geometry("260x180")

nivel = tk.StringVar(value="Intermediário")
tk.Label(j, text="Escolha o nível:",
        font=("Helvetica",11,"bold")).pack(anchor="w", padx=10, pady=6)
for n in ["Iniciante","Intermediário","Avançado"]:
    tk.Radiobutton(j, text=n, variable=nivel, value=n).pack(anchor="w", padx=14)

tk.Button(j, text="Confirmar",
        command=lambda: print("Selecionado:", nivel.get())).pack(pady=8)
j.mainloop()
```

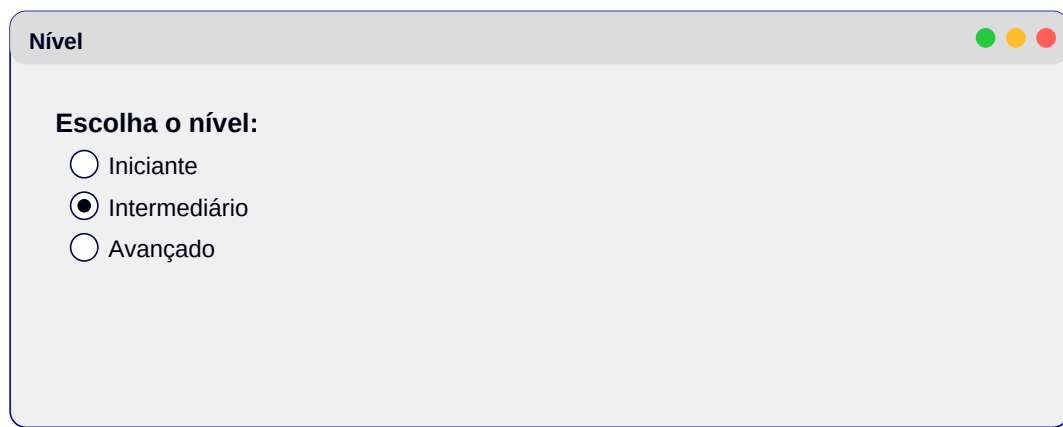


Figura 10.1 — Radiobutton com a opção “Intermediário” marcada.

11. Listbox

Exibe uma lista de itens selecionáveis. Métodos importantes: **insert('end', item)**, **delete(i)**, **get(i)**, **curselection()**, **selection_set(i)**.

Use **selectmode='single'** (padrão), 'browse', 'multiple' ou 'extended'.

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Frutas"); j.geometry("280x240")

barra = tk.Scrollbar(j); barra.pack(side="right", fill="y")
lst = tk.Listbox(j, yscrollcommand=barra.set, font=("Helvetica",11),
                 selectbackground="#0078D4", selectforeground="white")
for f in ["Maçã","Banana","Laranja","Uva","Abacaxi","Manga","Pera"]:
    lst.insert("end", f)
lst.pack(fill="both", expand=True, padx=8, pady=8)
barra.config(command=lst.yview)
lst.selection_set(2)
j.mainloop()
```

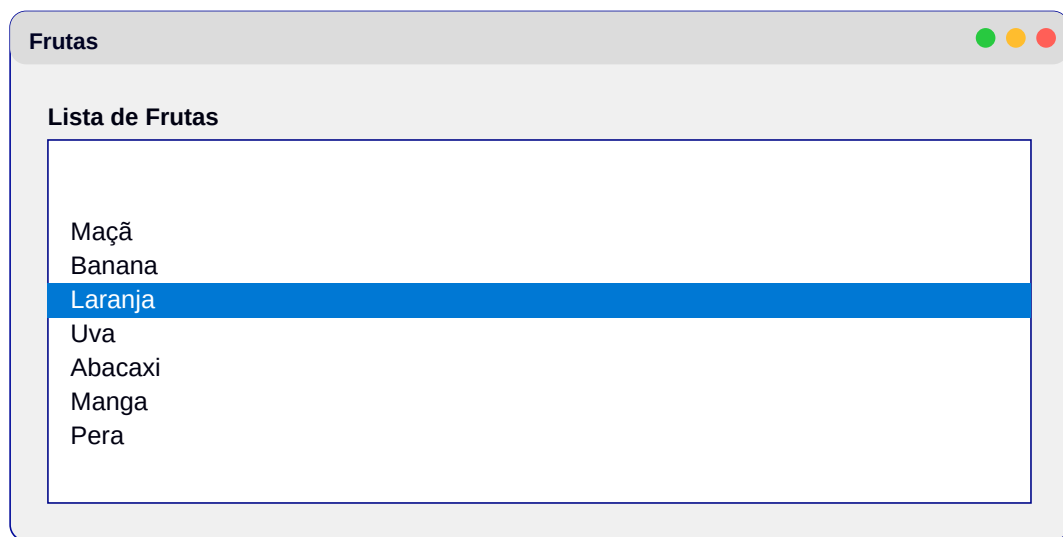


Figura 11.1 — Listbox com item “Laranja” selecionado.

12. ttk.Combobox

Caixa de seleção (dropdown). Faz parte do submódulo **ttk**, que oferece widgets temáticos modernos.

Parâmetros: **values** (lista), **state='readonly'** (impede digitação). Métodos: **get()**, **set(valor)**, evento '**<<ComboboxSelected>>**'.

Exemplo:

```
import tkinter as tk
from tkinter import ttk

j = tk.Tk(); j.title("Combobox"); j.geometry("320x140")
tk.Label(j, text="Linguagem favorita:",
        font=("Helvetica",11,"bold")).pack(anchor="w", padx=10, pady=8)

cb = ttk.Combobox(j, state="readonly",
                 values=["Python","JavaScript","Java","C++","Go"])
cb.current(0)
cb.pack(fill="x", padx=10)
cb.bind("<<ComboboxSelected>>",
       lambda e: print("Escolheu:", cb.get()))
j.mainloop()
```

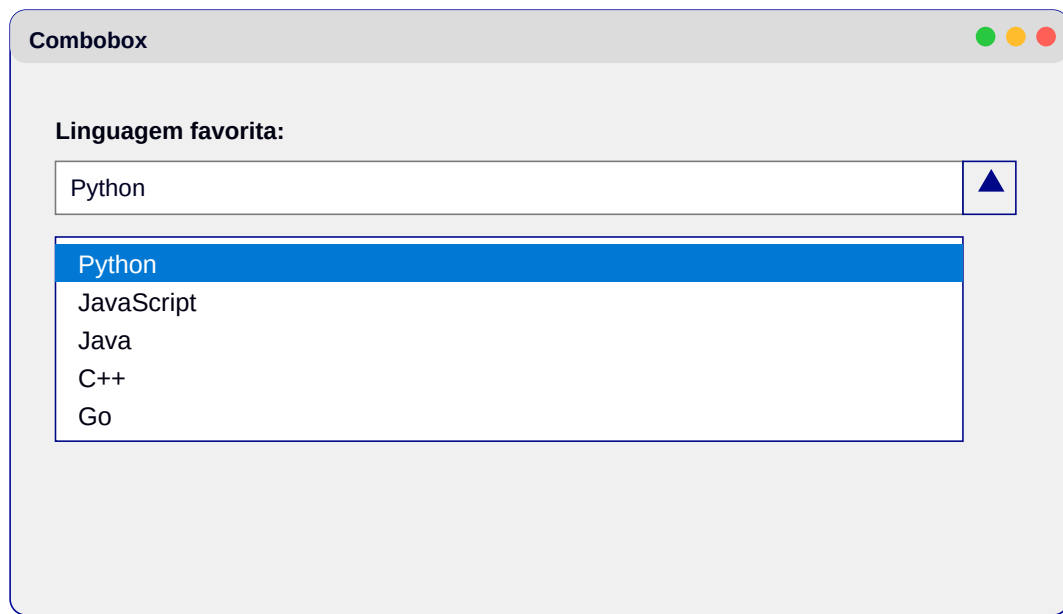


Figura 12.1 — Combobox aberto exibindo as opções.

13. Scale (slider)

Controle deslizante para escolher um número dentro de uma faixa. Parâmetros: **from_**, **to**, **orient** ('horizontal'/'vertical'), **resolution**, **tickinterval**, **command**.

Exemplo:

```
import tkinter as tk
def atualizar(v):
    lbl.config(text=f"Valor: {int(float(v))}")

j = tk.Tk(); j.title("Volume"); j.geometry("340x140")
tk.Label(j, text="Volume:", font=("Helvetica",11,"bold"))
    .pack(anchor="w", padx=10, pady=4)
tk.Scale(j, from_=0, to=100, orient="horizontal",
        command=atualizar, length=300).pack(padx=10)
lbl = tk.Label(j, text="Valor: 0",
        font=("Helvetica",12,"bold"))
lbl.pack(pady=4)
j.mainloop()
```

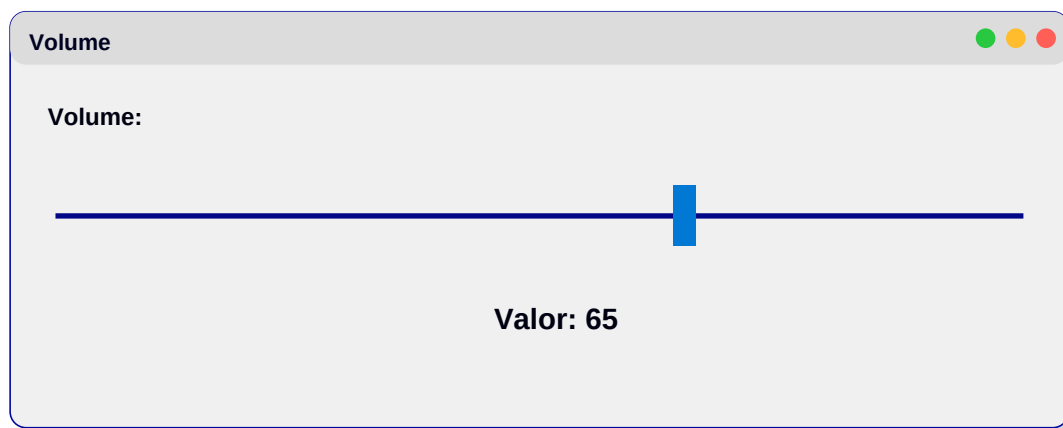


Figura 13.1 — Scale horizontal de 0 a 100.

14. Spinbox

Campo numérico com setas para incremento/decremento. Parâmetros: **from_**, **to**, **increment**, **values**, **wrap** (se True circula nas extremidades).

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Spinbox"); j.geometry("260x130")
tk.Label(j, text="Quantidade:",
        font=("Helvetica",11,"bold")).pack(pady=6)
sp = tk.Spinbox(j, from_=1, to=99, increment=1,
               font=("Helvetica",14), width=6, justify="center")
sp.pack()
tk.Button(j, text="Confirmar",
        command=lambda: print("Qtd:", sp.get())).pack(pady=10)
j.mainloop()
```

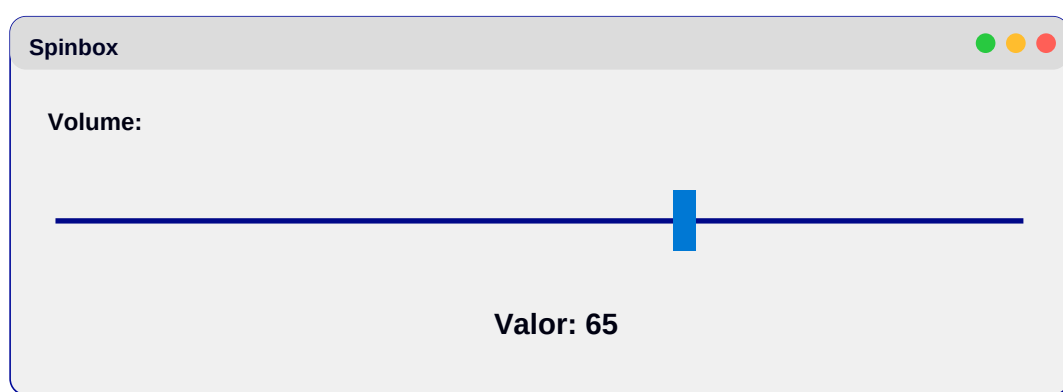


Figura 14.1 — Spinbox controlando uma quantidade.

15. Scrollbar

A Scrollbar nunca é usada sozinha: ela controla a rolagem de outro widget (Text, Listbox, Canvas, Treeview).

Padrão: crie a barra, conecte o widget com `yscrollcommand=barra.set` e a barra com `barra.config(command=widget.yview)`.

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Scrollbar"); j.geometry("260x220")
barra = tk.Scrollbar(j); barra.pack(side="right", fill="y")
lst = tk.Listbox(j, yscrollcommand=barra.set)
for i in range(1, 51): lst.insert("end", f"Item {i}")
lst.pack(side="left", fill="both", expand=True)
barra.config(command=lst.yview)
j.mainloop()
```

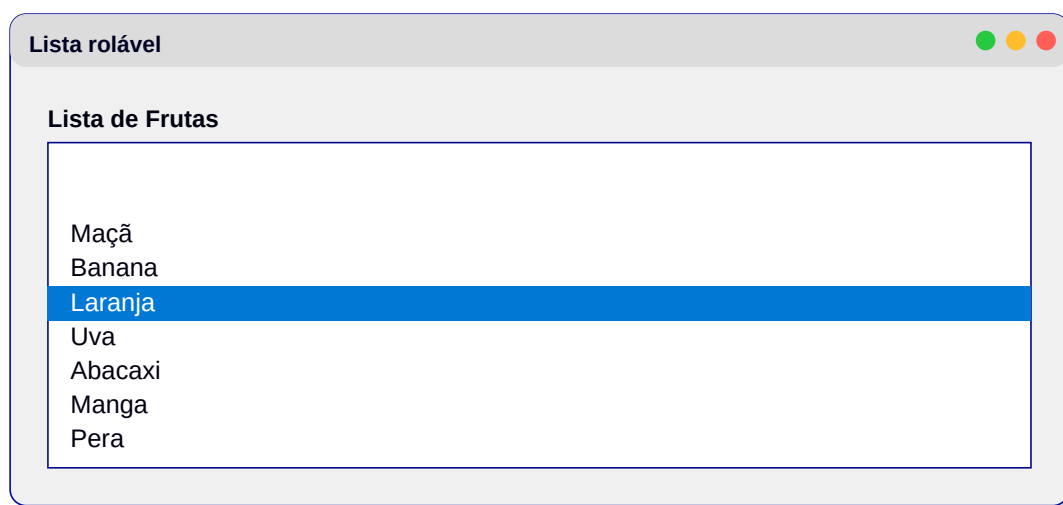


Figura 15.1 — Listbox com Scrollbar vertical.

16. Canvas

O **Canvas** é a área de desenho do Tkinter. Permite desenhar linhas, retângulos, círculos, polígonos, textos e imagens — e ainda capturar cliques sobre cada objeto.

Métodos principais: **create_line**, **create_rectangle**, **create_oval**, **create_polygon**, **create_text**, **create_image**. Cada um retorna um *id* para depois mover (**coords**, **move**) ou apagar (**delete**).

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Canvas"); j.geometry("480x260")

c = tk.Canvas(j, bg="white"); c.pack(fill="both", expand=True)
c.create_oval(40, 60, 140, 160, fill="#E91E63", outline="")
c.create_rectangle(180, 60, 280, 160, fill="#3F51B5", outline="")
c.create_line(320, 60, 420, 200, fill="#4CAF50", width=3)
c.create_polygon(440, 60, 480, 200, 400, 200, fill="#FF9800")
c.create_text(240, 230, text="Canvas em ação",
             font=("Helvetica", 12, "bold"))
j.mainloop()
```

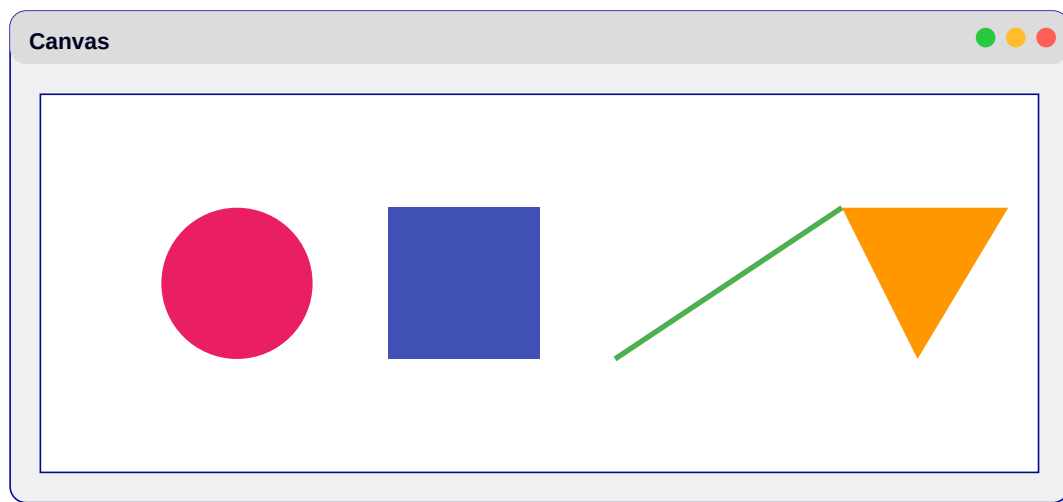


Figura 16.1 — Formas básicas desenhadas no Canvas.

17. Menu e Menubar

Para criar um menu na barra de título: instancie **Menu(janela)**, defina como `janela.config(menu=mb)` e adicione cascatas com **add_cascade**.

Submenus aceitam: **add_command(label,command)**, **add_separator()**, **add_checkbutton**, **add_radiobutton**.

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("App com menu"); j.geometry("420x220")

mb = tk.Menu(j); j.config(menu=mb)
arq = tk.Menu(mb, tearoff=0)
arq.add_command(label="Novo",      command=lambda: print("Novo"))
arq.add_command(label="Abrir...",  command=lambda: print("Abrir"))
arq.add_command(label="Salvar",    command=lambda: print("Salvar"))
arq.add_command(label="Salvar como...", command=lambda: print("SalvarC"))
arq.add_separator()
arq.add_command(label="Sair", command=j.destroy)
mb.add_cascade(label="Arquivo", menu=arq)
for m in ["Editar", "Exibir", "Ajuda"]:
    mb.add_cascade(label=m, menu=tk.Menu(mb, tearoff=0))
j.mainloop()
```

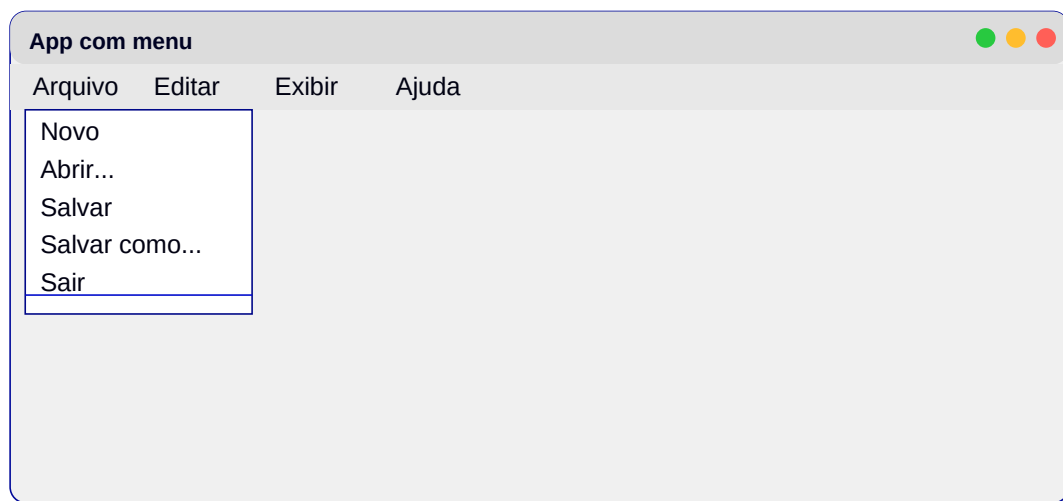


Figura 17.1 — Menu “Arquivo” expandido com separador.

18. messagebox

Submódulo **tkinter.messagebox** com diálogos prontos: **showinfo**, **showwarning**, **showerror**, **askyesno**, **askokcancel**, **askretrycancel**, **askquestion**.

Os métodos com **ask** retornam **True/False** (ou strings como 'yes'/'no').

Exemplo:

```
import tkinter as tk
from tkinter import messagebox

j = tk.Tk(); j.title("Diálogos"); j.geometry("260x140")

def perguntar():
    if messagebox.askyesno("Confirmação", "Deseja realmente sair?"):
        j.destroy()

tk.Button(j, text="Info",
        command=lambda: messagebox.showinfo("Info", "Operação concluída"))
.pack(pady=4)
tk.Button(j, text="Erro",
        command=lambda: messagebox.showerror("Erro", "Falha ao salvar"))
.pack(pady=4)
tk.Button(j, text="Sair", command=perguntar).pack(pady=4)
j.mainloop()
```

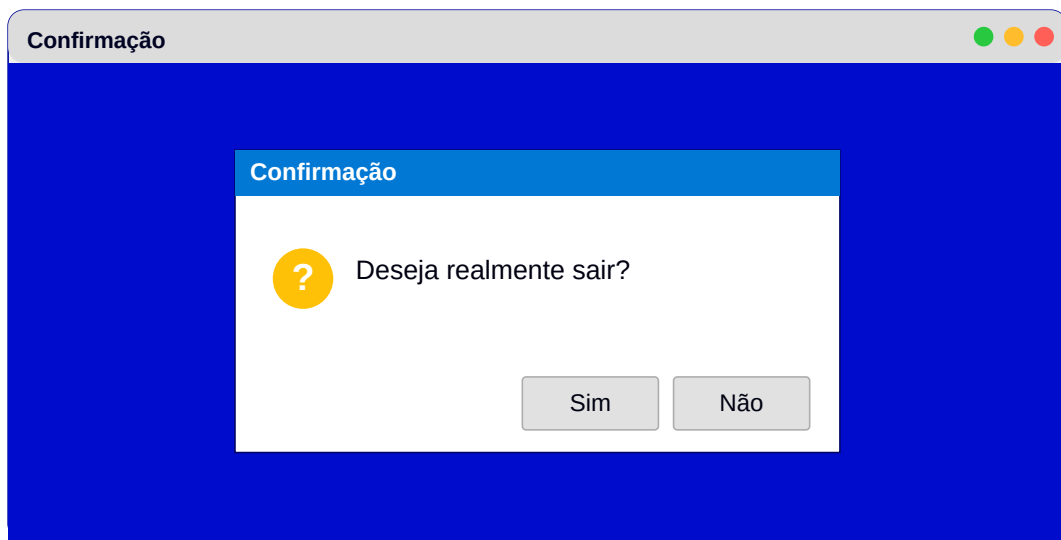


Figura 18.1 — Caixa de confirmação askyesno.

19. filedialog e colorchooser

tkinter.filedialog abre janelas nativas para escolher arquivos/pastas: **askopenfilename**, **asksaveasfilename**, **askdirectory**, **askopenfilenames** (múltiplos).

tkinter.colorchooser.askcolor() abre o seletor de cores e retorna uma tupla ((r,g,b), '#rrggbb').

Exemplo:

```
import tkinter as tk
from tkinter import filedialog, colorchooser

j = tk.Tk(); j.title("Diálogos avançados"); j.geometry("320x180")

def abrir():
    cam = filedialog.askopenfilename(
        title="Selecione um arquivo",
        filetypes=[("PDF", "*.pdf"), ("Todos", "*.")]
    )
    print("Arquivo:", cam)

def cor():
    c = colorchooser.askcolor(title="Escolher cor")
    j.configure(bg=c[1])

tk.Button(j, text="Abrir arquivo", command=abrir).pack(pady=10)
tk.Button(j, text="Escolher cor de fundo", command=cor).pack(pady=10)
j.mainloop()
```

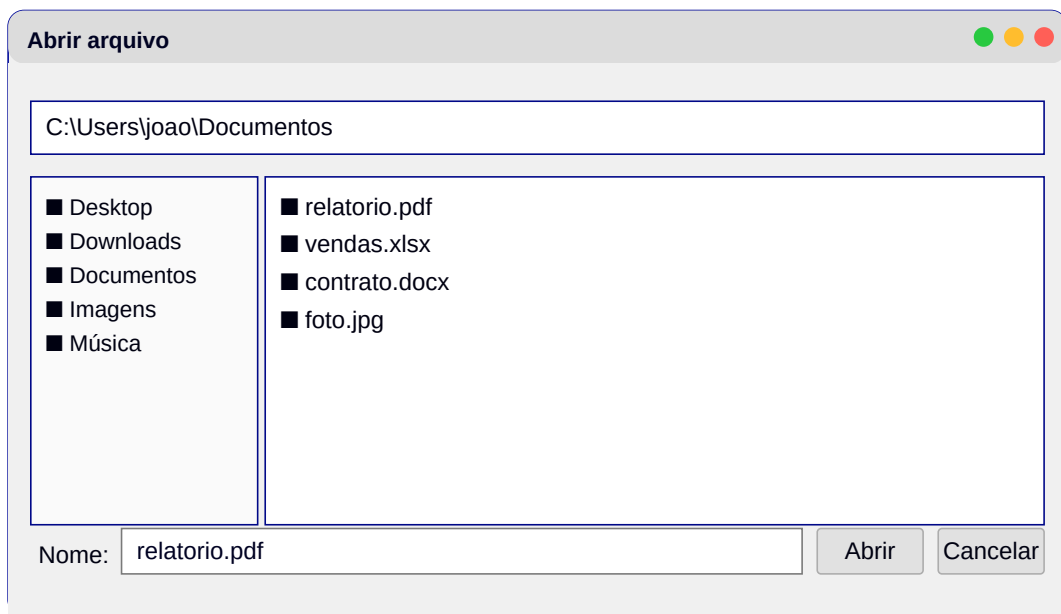


Figura 19.1 — Janela do askopenfilename().

20. Toplevel — múltiplas janelas

Para abrir janelas secundárias use **Toplevel(janela_pai)**. Ela funciona como uma Tk independente, mas mantém o vínculo com a janela principal.

Métodos úteis: **transient(pai)** (deixa ficar sempre acima do pai), **grab_set()** (modal — bloqueia interação com o pai) e **wait_window()**.

Exemplo:

```
import tkinter as tk

def abrir_sobre():
    top = tk.Toplevel(j); top.title("Sobre")
    top.geometry("280x140"); top.transient(j); top.grab_set()
    tk.Label(top, text="Meu App v1.0",
             font=("Helvetica",13,"bold")).pack(pady=10)
    tk.Label(top, text="Desenvolvido em Tkinter").pack()
    tk.Button(top, text="OK", command=top.destroy).pack(pady=10)

j = tk.Tk(); j.title("Principal"); j.geometry("320x180")
tk.Button(j, text="Sobre...", command=abrir_sobre).pack(pady=40)
j.mainloop()
```

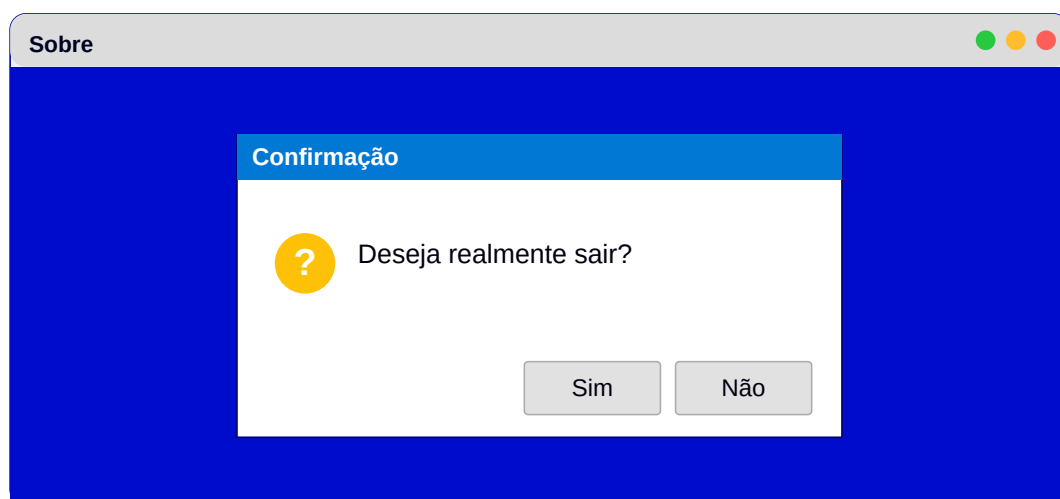


Figura 20.1 — Janela modal Toplevel “Sobre”.

21. ttk.Treeview

O Treeview do ttk é um widget poderoso para exibir **tabelas** e **árvores hierárquicas**.

Use **columns=('a','b','c')** e **show='headings'** para tabela. Configure cada coluna com **heading()** e **column()**. Insira linhas com **insert("", 'end', values=(...))**.

Exemplo:

```
import tkinter as tk
from tkinter import ttk

j = tk.Tk(); j.title("Funcionários"); j.geometry("520x260")

tv = ttk.Treeview(j, columns=("id", "nome", "cargo", "salario"),
                  show="headings", height=8)
for c, txt, w in [("id", "ID", 40), ("nome", "Nome", 120),
                  ("cargo", "Cargo", 160), ("salario", "Salário", 100)]:
    tv.heading(c, text=txt); tv.column(c, width=w, anchor="w")

dados = [(1, "Ana", "Desenvolvedora", "R$ 8.500"),
          (2, "Carlos", "Designer", "R$ 6.200"),
          (3, "Beatriz", "Gerente", "R$ 12.000"),
          (4, "Diego", "Analista", "R$ 5.800")]
for d in dados: tv.insert("", "end", values=d)
tv.pack(fill="both", expand=True, padx=10, pady=10)
j.mainloop()
```



ID	Nome	Cargo	Salário
1	Ana	Desenvolvedora	R\$ 8.500
2	Carlos	Designer	R\$ 6.200
3	Beatriz	Gerente	R\$ 12.000
4	Diego	Analista	R\$ 5.800

Figura 21.1 — Treeview funcionando como tabela.

22. ttk.Notebook

Notebook cria abas (tabs). Cada aba é um Frame adicionado com **add(frame, text='Título')**.

Exemplo:

```
import tkinter as tk
from tkinter import ttk

j = tk.Tk(); j.title("Configurações"); j.geometry("420x260")
nb = ttk.Notebook(j); nb.pack(fill="both", expand=True, padx=8, pady=8)

f1 = ttk.Frame(nb); f2 = ttk.Frame(nb); f3 = ttk.Frame(nb)
nb.add(f1, text="Geral"); nb.add(f2, text="Aparência"); nb.add(f3, text="Avançado")

ttk.Label(f2, text="Tema da aplicação:",
          font=("Helvetica",11,"bold")).pack(anchor="w", padx=10, pady=8)
tema = tk.StringVar(value="Escuro")
for t in ["Claro","Escuro","Sistema"]:
    ttk.Radiobutton(f2, text=t, value=t, variable=tema).pack(anchor="w", padx=14)
j.mainloop()
```

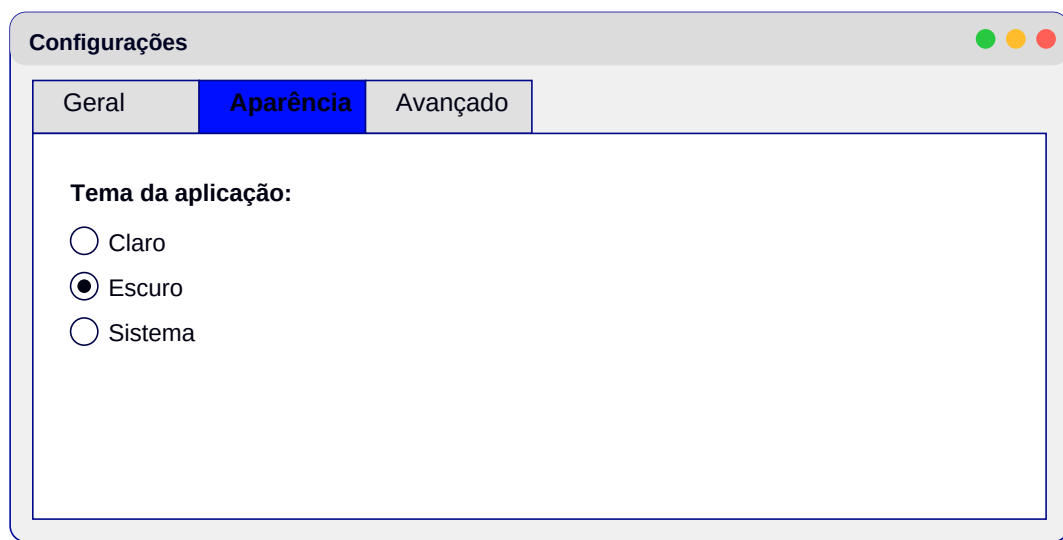


Figura 22.1 — Notebook com a aba “Aparência” ativa.

23. ttk.Progressbar

Mostra progresso de uma tarefa. **mode**='determinate' (com valor 0–100) ou 'indeterminate' (animação contínua).

Métodos: **start(ms)**, **stop()**, **step(qtd)**. Atribua o valor com `pb['value']=n`.

Exemplo:

```
import tkinter as tk
from tkinter import ttk

j = tk.Tk(); j.title("Download"); j.geometry("360x160")
tk.Label(j, text="Baixando arquivo...",
        font=("Helvetica",11,"bold")).pack(pady=6)
pb = ttk.Progressbar(j, length=300, mode="determinate", maximum=100)
pb.pack(pady=8); pb['value'] = 62
lbl = tk.Label(j, text="62% concluído"); lbl.pack()

def passo():
    if pb['value'] < 100:
        pb['value'] += 2
        lbl.config(text=f"{int(pb['value'])}% concluído")
        j.after(80, passo)
passo()
j.mainloop()
```

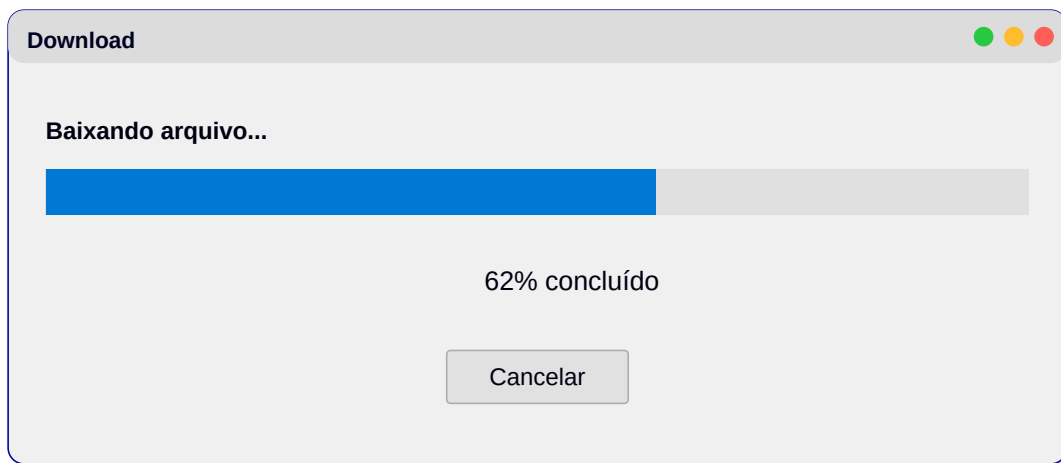


Figura 23.1 — Barra de progresso em 62%.

24. Variáveis Tk

São objetos especiais que mantêm sincronia automática entre o widget e a variável Python. Tipos: **StringVar**, **IntVar**, **DoubleVar**, **BooleanVar**.

Métodos principais: **get()**, **set(valor)**, **trace_add('write', callback)** (executa callback quando o valor muda).

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("StringVar"); j.geometry("320x160")

nome = tk.StringVar(value="Mundo")
tk.Label(j, text="Digite seu nome:").pack(pady=6)
tk.Entry(j, textvariable=nome).pack()

saida = tk.Label(j, text="", font=("Helvetica",13,"bold"))
saida.pack(pady=12)

nome.trace_add("write",
    lambda *a: saida.config(text=f"Olá, {nome.get()}!"))
nome.set("Mundo")
j.mainloop()
```

25. Eventos e bindings

Use **widget.bind(<Evento>, funcao)** para reagir a teclado, mouse e foco.

Eventos comuns: <Button-1> (clique esquerdo), <Double-Button-1>, <Enter>, <Leave>, <Key>, <Return>, <MouseWheel>.

Exemplo:

```
import tkinter as tk
j = tk.Tk(); j.title("Eventos"); j.geometry("320x180")

lbl = tk.Label(j, text="Mova o mouse / digite algo",
               font=("Helvetica",11)); lbl.pack(pady=20)

def mover(e): lbl.config(text=f"Mouse em ({e.x},{e.y})")
def tecla(e): lbl.config(text=f"Tecla: {e.keysym}")

j.bind("<Motion>", mover)
j.bind("<Key>", tecla)
j.mainloop()
```

26. Estilos com ttk.Style

Você pode aplicar temas e personalizar widgets ttk via **ttk.Style**.

Métodos: **theme_use('clam'), configure('TButton', font=...), map('TButton', background=[('active','red')])**.

Exemplo:

```
import tkinter as tk
from tkinter import ttk
j = tk.Tk(); j.title("ttk Style"); j.geometry("300x180")
style = ttk.Style(); style.theme_use("clam")
style.configure("Big.TButton", font=("Helvetica",12,"bold"),
                foreground="white", background="#0078D4", padding=8)
style.map("Big.TButton", background=[("active","#005A9E")])
ttk.Button(j, text="Botão estilizado",
           style="Big.TButton").pack(pady=40)
j.mainloop()
```

Parte II — CustomTkinter

27. Instalação e CTK()

CustomTkinter é uma biblioteca moderna construída sobre o Tkinter. Ela oferece widgets com aparência atual (cantos arredondados, modo escuro, temas), mantendo a mesma API básica.

Instalação: `pip install customtkinter`.

A janela principal é **customtkinter.CTk()** (substitui `tk.Tk()`). Quase todos os widgets do Tkinter têm equivalente prefixado por **CTk**.

Exemplo:

```
import customtkinter as ctk

ctk.set_appearance_mode("dark")      # "light", "dark" ou "system"
ctk.set_default_color_theme("blue")  # "blue", "green", "dark-blue"

app = ctk.CTk()
app.title("Meu app moderno")
app.geometry("400x220")

ctk.CTkLabel(app, text="Olá CustomTkinter!",
              font=("Helvetica", 18, "bold")).pack(pady=20)
ctk.CTkButton(app, text="Clique aqui",
               command=lambda: print("Clicado!")).pack(pady=10)
app.mainloop()
```

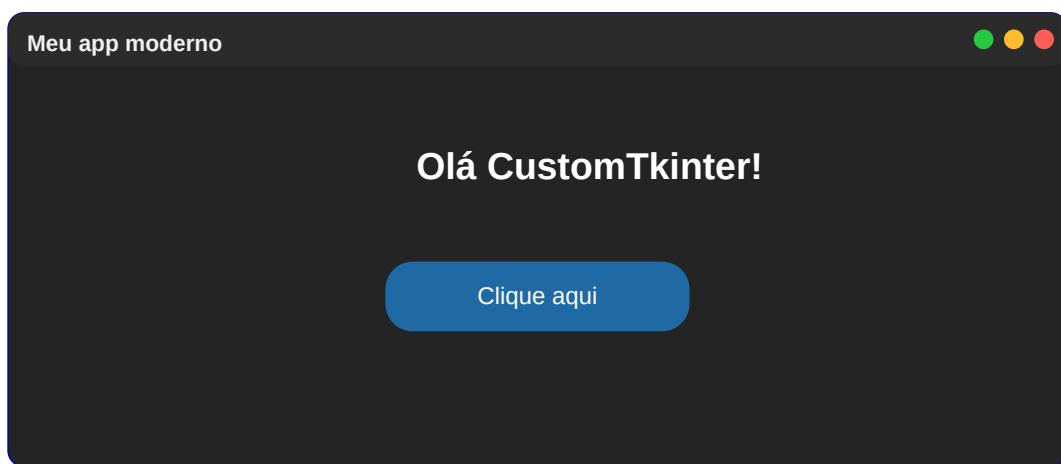


Figura 27.1 — Primeira janela com CustomTkinter (modo escuro).

28. Aparência e temas

Funções globais para personalizar a aparência:

set_appearance_mode('light' / 'dark' / 'system') — define modo claro/escuro/seguir o SO.

set_default_color_theme('blue' / 'green' / 'dark-blue') — define a cor de destaque global.

set_widget_scaling(1.1) e **set_window_scaling(1.0)** — ajustam escala (útil para acessibilidade).

Exemplo:

```
import customtkinter as ctk
ctk.set_appearance_mode("system")
ctk.set_default_color_theme("green")
ctk.set_widget_scaling(1.05)

app = ctk.CTk(); app.geometry("360x180"); app.title("Tema")

def alternar():
    novo = "dark" if ctk.get_appearance_mode()=="Light" else "light"
    ctk.set_appearance_mode(novo)

ctk.CTkLabel(app, text="Alterne o tema").pack(pady=20)
ctk.CTkButton(app, text="Alternar tema",
               command=alternar).pack()
app.mainloop()
```

29. CTkLabel e CTkButton

Parâmetros comuns aos widgets CTk: **text**, **font**, **text_color**, **fg_color** (fundo), **hover_color**, **corner_radius**, **border_width**, **width**, **height**, **command**.

Cores podem ser nomes ("transparent") ou tuplas ("cor_clara","cor_escura") — automaticamente escolhe a correta segundo o tema.

Exemplo:

```
import customtkinter as ctk
app = ctk.CTk(); app.title("Botões"); app.geometry("360x220")
ctk.set_appearance_mode("dark")

ctk.CTkLabel(app, text="Painel de controle",
             font=("Helvetica",16,"bold")).pack(pady=12)

ctk.CTkButton(app, text="Salvar",
             fg_color="#1F6AA5", hover_color="#155A8A",
             corner_radius=10, width=200,
             command=lambda: print("salvar")).pack(pady=6)

ctk.CTkButton(app, text="Cancelar",
             fg_color="transparent", border_width=2,
             text_color=("gray10","gray90"),
             command=app.destroy).pack(pady=6)
app.mainloop()
```

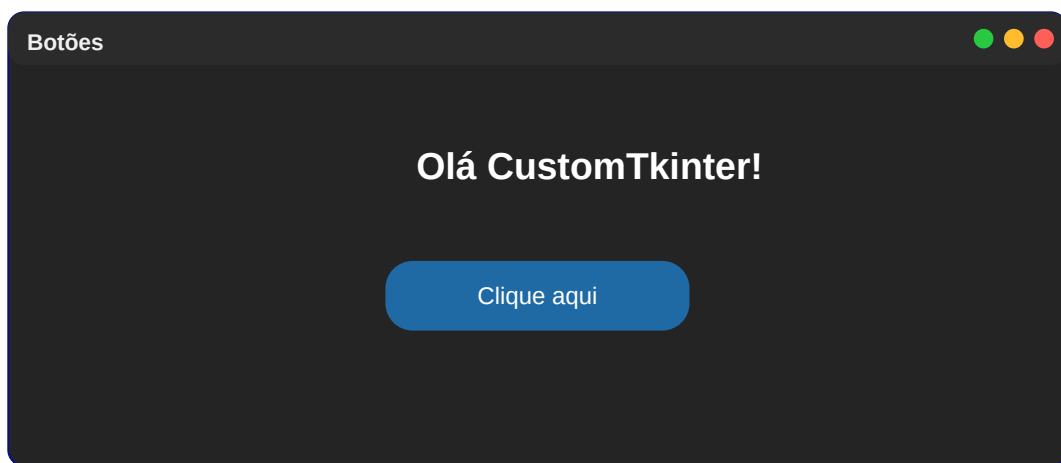


Figura 29.1 — Botões com `fg_color`, `hover_color` e `corner_radius`.

30. CTkEntry e CTkTextbox

CTkEntry — campo de uma linha. Parâmetros extras: **placeholder_text** (texto cinza enquanto vazio), **show='*'** para senhas.

CTkTextbox — múltiplas linhas com scrollbar embutida. Métodos iguais ao Text: insert/get/delete.

Exemplo:

```
import customtkinter as ctk
app = ctk.CTk(); app.title("Login"); app.geometry("360x260")

ctk.CTkLabel(app, text="Login",
              font=("Helvetica", 18, "bold")).pack(pady=10)

usr = ctk.CTkEntry(app, placeholder_text="Usuário", width=260)
usr.pack(pady=6)
pwd = ctk.CTkEntry(app, placeholder_text="Senha",
                  show="•", width=260)
pwd.pack(pady=6)

ctk.CTkButton(app, text="Entrar", width=260,
               command=lambda: print(usr.get(), pwd.get())).pack(pady=10)
ctk.CTkLabel(app, text="Esqueci minha senha",
              text_color="gray").pack()
app.mainloop()
```

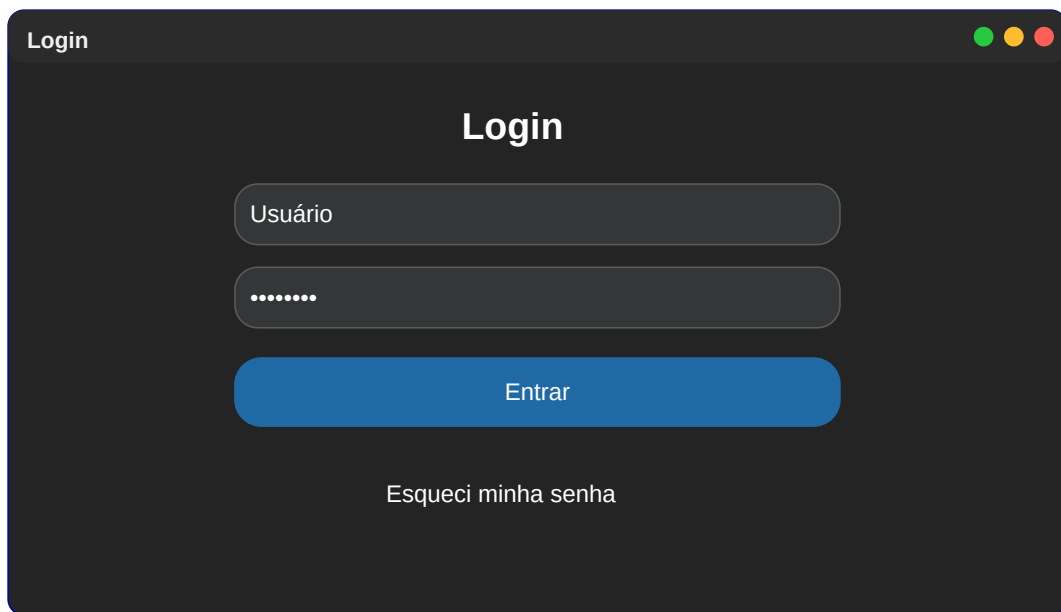


Figura 30.1 — Formulário de login com CTkEntry e placeholder.

31. CTkFrame e CTkScrollableFrame

CTkFrame é o container. Pode receber **corner_radius** e **fg_color**, criando “cards” modernos.

CTkScrollableFrame é um frame com barra de rolagem automática — perfeito para listas longas dentro de uma tela.

Exemplo:

```
import customtkinter as ctk
app = ctk.CTk(); app.geometry("360x300"); app.title("Lista rolável")

frame = ctk.CTkScrollableFrame(app, label_text="Tarefas",
                               width=320, height=260)
frame.pack(padx=10, pady=10, fill="both", expand=True)
for i in range(1, 31):
    ctk.CTkCheckBox(frame, text=f"Tarefa #{i}").pack(anchor="w", pady=2)
app.mainloop()
```

32. CTkCheckBox e CTkRadioButton

Funcionam como no Tkinter, mas com aparência moderna. Use **variable** (uma **tk.IntVar** / **StringVar**) para vincular.

Exemplo:

```
import customtkinter as ctk
import tkinter as tk
app = ctk.CTk(); app.geometry("320x220"); app.title("Preferências")

aceitar = tk.IntVar()
ctk.CTkCheckBox(app, text="Aceito os termos",
                variable=aceitar).pack(pady=10, padx=20, anchor="w")

plano = tk.StringVar(value="pro")
for t,v in [("Free","free"),("Pro","pro"),("Business","biz")]:
    ctk.CTkRadioButton(app, text=t, value=v,
                       variable=plano).pack(pady=4, padx=20, anchor="w")

app.mainloop()
```

33. CTkSwitch

Interruptor estilo iOS/Android. Igual ao CheckBox em comportamento, porém com visual moderno.

Exemplo:

```
import customtkinter as ctk
import tkinter as tk

app = ctk.CTk(); app.geometry("340x200"); app.title("Configurações")
escuro = tk.IntVar(value=1); notif = tk.IntVar(value=0)

ctk.CTkLabel(app, text="Modo Escuro:").pack(anchor="w", padx=20, pady=6)
ctk.CTkSwitch(app, text="", variable=escuro,
               command=lambda: ctk.set_appearance_mode(
                   "dark" if escuro.get() else "light")
               ).pack(padx=20, anchor="w")

ctk.CTkLabel(app, text="Notificações:").pack(anchor="w", padx=20, pady=6)
ctk.CTkSwitch(app, text="", variable=notif).pack(padx=20, anchor="w")
app.mainloop()
```

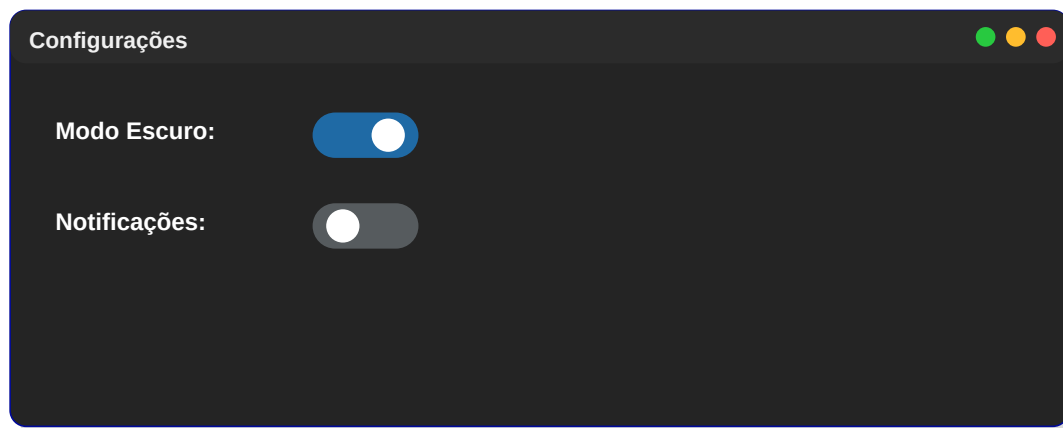


Figura 33.1 — Switch ligado (azul) e desligado (cinza).

34. CTKSlider e CTKProgressBar

CTKSlider — equivalente moderno do Scale. Parâmetros: **from_**, **to**, **number_of_steps**, **command**.

CTKProgressBar — recebe valores entre 0 e 1. Métodos: **set(valor)**, **start()**, **stop()**, **configure(mode='indeterminate')**.

Exemplo:

```
import customtkinter as ctk
app = ctk.CTk(); app.geometry("360x220"); app.title("Brilho / Progresso")

ctk.CTkLabel(app, text="Brilho:").pack(anchor="w", padx=12, pady=4)
val = ctk.CTkLabel(app, text="55%",
                    font=("Helvetica",14,"bold")); val.pack()
def att(v): val.configure(text=f"{int(v)}%")
ctk.CTkSlider(app, from_=0, to=100, number_of_steps=100,
              command=att).pack(fill="x", padx=12, pady=4)

ctk.CTkLabel(app, text="Processando...").pack(anchor="w", padx=12, pady=8)
pb = ctk.CTkProgressBar(app); pb.set(0.78); pb.pack(fill="x", padx=12)
app.mainloop()
```

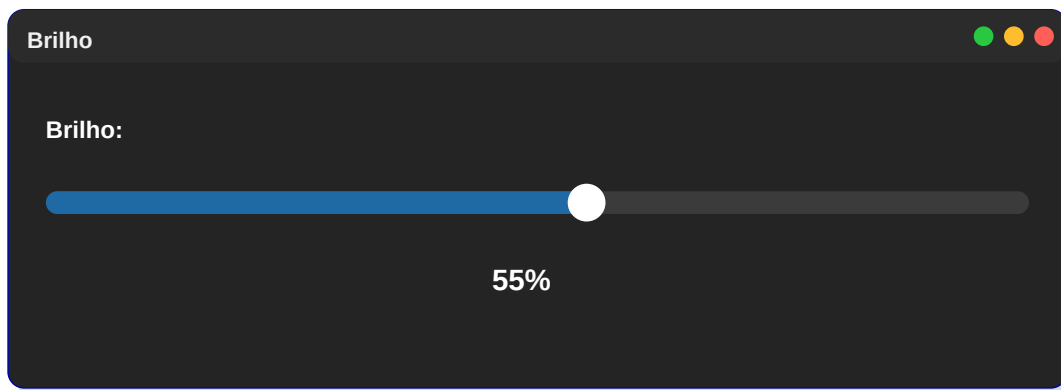


Figura 34.1 — CTKSlider com posição em 55%.

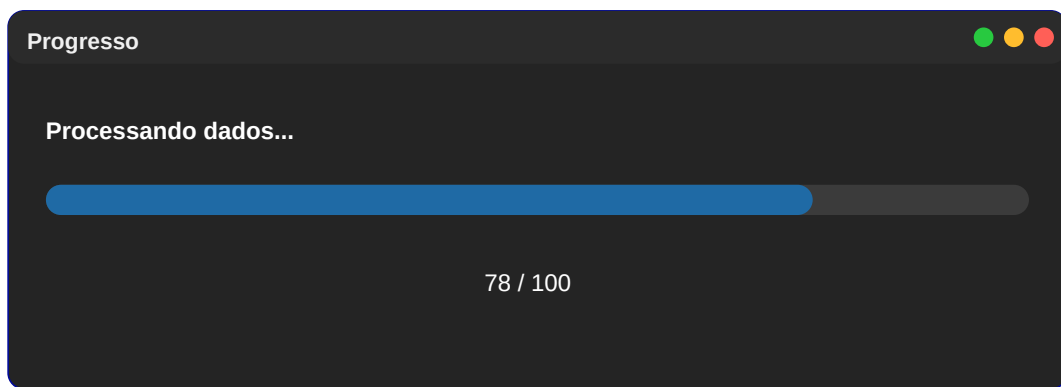


Figura 34.2 — CTKProgressBar em 78%.

35. CTkOptionMenu e CTkComboBox

Ambos são caixas de seleção. A diferença: **OptionMenu** só permite escolher dentre os valores, enquanto **ComboBox** também aceita digitação livre.

Exemplo:

```
import customtkinter as ctk
app = ctk.CTk(); app.geometry("320x200"); app.title("Selectors")

ctk.CTkLabel(app, text="Idioma:").pack(anchor="w", padx=12, pady=6)
ctk.CTkOptionMenu(app, values=["Português", "English", "Español"]
                  ).pack(padx=12, fill="x")

ctk.CTkLabel(app, text="Cidade:").pack(anchor="w", padx=12, pady=6)
ctk.CTkComboBox(app, values=["São Paulo", "Rio", "Curitiba"]
                ).pack(padx=12, fill="x")
app.mainloop()
```

36. CTkSegmentedButton e CTkTabview

CTkSegmentedButton — grupo de botões mutuamente exclusivos (como Radio, mas em formato horizontal).

CTkTabview — sistema de abas moderno. Adicione abas com **add('Nome')** e acesse com **tab('Nome')**.

Exemplo:

```
import customtkinter as ctk
app = ctk.CTk(); app.geometry("420x280"); app.title("Conta")

tv = ctk.CTkTabview(app); tv.pack(fill="both", expand=True, padx=10, pady=10)
for t in ["Perfil", "Conta", "Privacidade"]: tv.add(t)

f = tv.tab("Perfil")
ctk.CTkLabel(f, text="Nome completo:").pack(anchor="w", padx=10, pady=4)
ctk.CTkEntry(f, placeholder_text="Seu nome").pack(fill="x", padx=10)
ctk.CTkLabel(f, text="Bio:").pack(anchor="w", padx=10, pady=4)
ctk.CTkEntry(f, placeholder_text="Sobre você").pack(fill="x", padx=10)
app.mainloop()
```

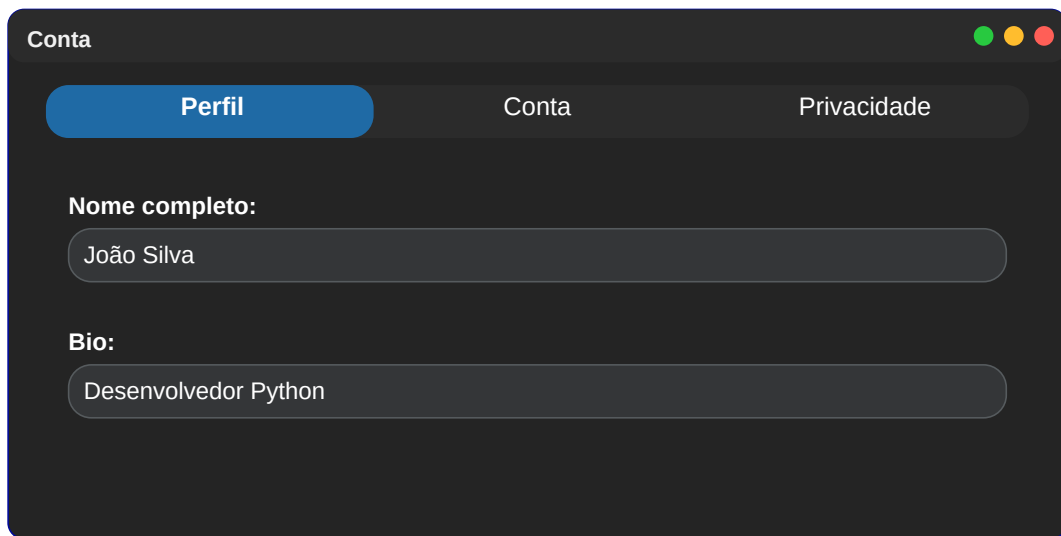


Figura 36.1 — CTkTabview com aba “Perfil” selecionada.

37. CTkImage

Permite carregar imagens com suporte a alta DPI e a dois temas (clara/escuro). Use junto com Pillow.

CTkImage(light_image=PIL_img, dark_image=PIL_img, size=(w,h)).

Exemplo:

```
import customtkinter as ctk
from PIL import Image

app = ctk.CTk(); app.geometry("320x220"); app.title("CTkImage")
img = ctk.CTkImage(
    light_image=Image.open("logo_claro.png"),
    dark_image=Image.open("logo_escuro.png"),
    size=(120,120))

ctk.CTkLabel(app, image=img, text="").pack(pady=20)
ctk.CTkLabel(app, text="Meu App").pack()
app.mainloop()
```

38. CTkToplevel

Equivalente moderno do Toplevel. Útil para janelas “Sobre”, configurações, popups personalizados.

CustomTkinter também oferece **CTkInputDialog(text='Pergunta', title='Input')** para pedir um texto rapidamente.

Exemplo:

```
import customtkinter as ctk

def abrir_sobre():
    t = ctk.CTkToplevel(app); t.title("Sobre"); t.geometry("260x140")
    ctk.CTkLabel(t, text="Meu App",
                 font=("Helvetica", 16, "bold")).pack(pady=10)
    ctk.CTkLabel(t, text="Versão 1.0 · Python GUI").pack()
    ctk.CTkButton(t, text="Fechar", command=t.destroy).pack(pady=10)

def perguntar_nome():
    d = ctk.CTkInputDialog(text="Qual seu nome?", title="Input")
    print("Nome:", d.get_input())

app = ctk.CTk(); app.geometry("320x180"); app.title("Diálogos")
ctk.CTkButton(app, text="Sobre...", command=abrir_sobre).pack(pady=14)
ctk.CTkButton(app, text="Pedir nome", command=perguntar_nome).pack()
app.mainloop()
```

39. Layout com grid em CTK

A mesma API **pack/grid/place** do Tkinter funciona aqui. Para layouts responsivos, configure pesos com **grid_columnconfigure(i, weight=1)** e **grid_rowconfigure(i, weight=1)**.

Use frames aninhados (cabeçalho, sidebar, conteúdo) e expanda só o que precisa crescer.

Exemplo:

```
import customtkinter as ctk
app = ctk.CTk(); app.geometry("520x300"); app.title("Layout")
app.grid_columnconfigure(1, weight=1)
app.grid_rowconfigure(0, weight=1)

side = ctk.CTkFrame(app, width=160)
side.grid(row=0, column=0, sticky="nsw", padx=8, pady=8)
ctk.CTkLabel(side, text="Menu",
             font=("Helvetica",14,"bold")).pack(pady=10)
for t in ["Início","Relatórios","Usuários"]:
    ctk.CTkButton(side, text=t, anchor="w").pack(fill="x", padx=8, pady=3)

main = ctk.CTkFrame(app)
main.grid(row=0, column=1, sticky="nsew", padx=8, pady=8)
ctk.CTkLabel(main, text="Conteúdo principal",
             font=("Helvetica",16,"bold")).pack(pady=40)
app.mainloop()
```

40. Projeto: Tela de Login moderna

Vamos juntar tudo num exemplo prático: uma tela de login com placeholder, validação e mensagem de erro.

Recursos usados: CTK, CTkFrame (card central), CTkEntry com placeholder, CTkButton com hover, CTkLabel para o título e para mensagem de status.

Exemplo:

```
import customtkinter as ctk
from tkinter import messagebox

ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

app = ctk.CTk(); app.geometry("420x420"); app.title("Login")

card = ctk.CTkFrame(app, corner_radius=14)
card.place(relx=0.5, rely=0.5, anchor="center", relwidth=0.82, relheight=0.82)

ctk.CTkLabel(card, text="Bem-vindo",
              font=("Helvetica", 22, "bold")).pack(pady=(20, 4))
ctk.CTkLabel(card, text="Acesse sua conta",
              text_color="gray").pack(pady=(0, 18))

usr = ctk.CTkEntry(card, placeholder_text="E-mail", width=260)
usr.pack(pady=6)
pwd = ctk.CTkEntry(card, placeholder_text="Senha",
                   show="•", width=260); pwd.pack(pady=6)

def entrar():
    if usr.get()=="admin@app.com" and pwd.get()=="123":
        messagebox.showinfo("Sucesso", "Login efetuado!")
    else:
        status.configure(text="Credenciais inválidas",
                        text_color="#FF5757")

ctk.CTkButton(card, text="Entrar", width=260,
               command=entrar).pack(pady=12)
status = ctk.CTkLabel(card, text=""); status.pack()
ctk.CTkLabel(card, text="Esqueci minha senha",
              text_color="gray").pack(pady=4)

app.mainloop()
```

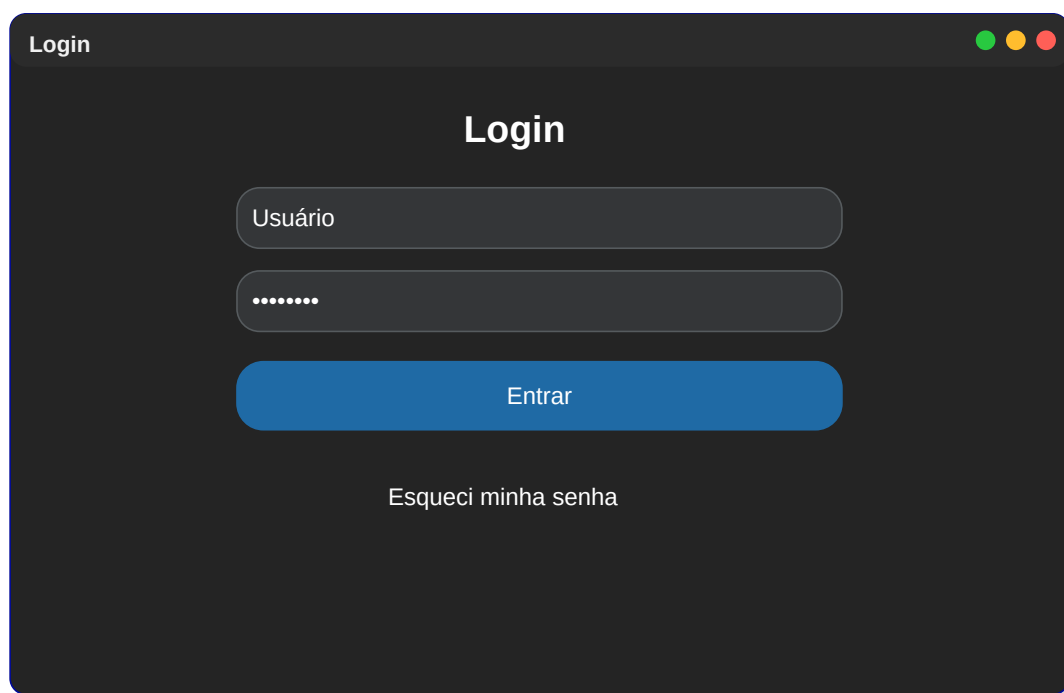


Figura 40.1 — Tela de Login finalizada.

41. Projeto: Dashboard completo

Aplicação típica com **sidebar de navegação**, **cards de KPI** no topo e um **gráfico** de exemplo (desenhado com Canvas).

Demonstra: grid responsivo, CtkFrame como card, organização visual em colunas, troca de aparência via Switch.

Exemplo:

```
import customtkinter as ctk
import tkinter as tk

ctk.set_appearance_mode("dark"); ctk.set_default_color_theme("blue")
app = ctk.CTk(); app.geometry("760x440"); app.title("Dashboard")
app.grid_columnconfigure(1, weight=1); app.grid_rowconfigure(0, weight=1)

# ---- Sidebar
sb = ctk.CTkFrame(app, width=180, corner_radius=0)
sb.grid(row=0, column=0, sticky="nsw")
ctk.CTkLabel(sb, text="■ Meu App",
             font=("Helvetica",16,"bold")).pack(pady=14)
for t in ["■ Início", "■ Relatórios", "■ Usuários",
         "■ Configurações", "■ Sair"]:
    ctk.CTkButton(sb, text=t, anchor="w",
                  fg_color="transparent",
                  hover_color="#1F6AA5")
    .pack(fill="x", padx=8, pady=2)

# ---- Conteúdo principal
main = ctk.CTkFrame(app, corner_radius=0)
main.grid(row=0, column=1, sticky="nsew")

ctk.CTkLabel(main, text="Dashboard",
             font=("Helvetica",20,"bold"))
    .grid(row=0, column=0, colspan=3, sticky="w",
          padx=16, pady=10)

# Cards de KPI
kpis = [("Usuários", "1.245"), ("Vendas", "R$ 87k"), ("Pedidos", "324")]
for i,(t,v) in enumerate(kpis):
    f = ctk.CTkFrame(main, height=80, corner_radius=10)
    f.grid(row=1, column=i, sticky="ew", padx=10, pady=4)
    main.grid_columnconfigure(i, weight=1)
    ctk.CTkLabel(f, text=t, text_color="gray")
        .pack(anchor="w", padx=10, pady=(8,0))
    ctk.CTkLabel(f, text=v,
                 font=("Helvetica",20,"bold"))
        .pack(anchor="w", padx=10, pady=(0,8))

# Gráfico de barras com Canvas
grafico = ctk.CTkFrame(main, corner_radius=10, height=200)
grafico.grid(row=2, column=0, colspan=3, sticky="nsew",
             padx=10, pady=10)
main.grid_rowconfigure(2, weight=1)
ctk.CTkLabel(grafico, text="Vendas mensais",
             font=("Helvetica",13,"bold"))
    .pack(anchor="w", padx=10, pady=6)

cv = tk.Canvas(grafico, bg="#2B2B2B", highlightthickness=0,
               height=150)
cv.pack(fill="both", expand=True, padx=10, pady=6)
valores = [30,50,40,70,60,85,55,90]
for i,v in enumerate(valores):
    cv.create_rectangle(20+i*60, 150-v*1.4, 70+i*60, 150,
                       fill="#1F6AA5", outline="")
```

```
app.mainloop()
```

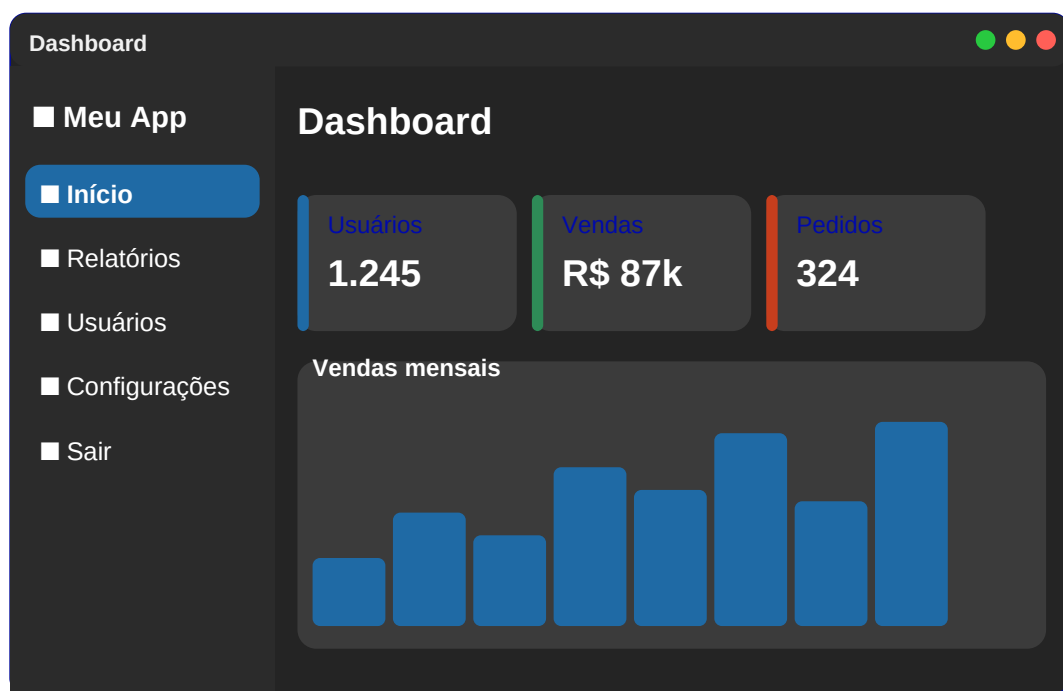


Figura 41.1 — Dashboard com sidebar, KPIs e gráfico.

42. Boas práticas e dicas finais

- **Organize em classes.** A partir de telas médias, herde de `tk.Tk` ou `ctk.CTk` e crie métodos para cada parte da interface — facilita a manutenção.
- **Separe lógica e UI.** Coloque regras de negócio em módulos próprios; a janela só chama funções.
- **Cuidado com o mainloop.** Operações longas travam a interface. Use `after()` para agendar tarefas, ou rode em **threads** e atualize a UI somente na thread principal.
- **Evite cores hardcoded em CustomTkinter.** Use tuplas `("clara","escura")` para que tudo funcione bem nos dois modos.
- **Use ttk.Style ou temas do CTK.** Padronize cores, fontes e espaçamento desde o início — economiza tempo depois.
- **Sempre teste o redimensionamento.** Use `grid` com pesos ou `pack(fill, expand)` para layouts responsivos.
- **Mantenha o usuário informado.** Use `Progressbar`, mensagens com `messagebox` e estados de botão `("disabled")` durante operações.
- **Atalhos de teclado.** Vincule `<Return>` ao botão padrão e `<Escape>` ao Cancelar.
- **Distribuição.** Para entregar como executável use **PyInstaller**: `pyinstaller --noconsole --onefile app.py`.

Conclusão

Você passou pelos principais widgets do Tkinter e do CustomTkinter, viu exemplos completos com código e a representação visual de cada tela. Combine esses blocos para construir desde utilitários simples até dashboards profissionais. Pratique recriando os exemplos do zero — é a melhor forma de fixar.

Bons códigos! ■